Text Classification, Feature Selection and Sentiment Analysis

IFN647 – Week 9

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

Learning Objectives

Topic ID	Key topics	Mapping to slides	Review Questions
1	Text Classification	2-6	
2	Feature Selection & Evaluation	7-12	
3	Vector Space Classifiers	13-20	Q3
4	Generative Model - Naïve Bayes Classifier	21-31	Q1, Q2
5	Sentiment Analysis	32-36	Q4
6	Methodologies for Sentiment Analysis	37-39	Q5
7	Sentiment Analysis in the Era of LLMs	40-42	Q5
	References	43	

1. Text Classification

- Text classification is to assign a document to one or more classes or categories.
  - The documents to be classified may be texts, images, music, etc.
  - When not otherwise specified, text classification is implied.
- Documents may be classified according to their **subjects (contents) or other attributes (or meta-data**, such as document type, author, printing year etc.)
  - The content-based approach
    - The class assigned to a document is based on subjects (topics) in the document.
  - The request-based approach
    - The class assigned to a document is based on “relevance”, i.e., the anticipated request from users is influencing how documents are being classified. It is targeted towards a particular audience or user group.
  - The statistical approaches for categorization
    - It uses machine learning methods to learn automatic classification rules based on human-labelled training documents.

Machine Learning for Text Classification

- Machine Learning Methods
  - Supervised learning - infers a classifier or functions from labelled training data.
    - E.g., Classification
    - Asks “what class does this item (document) belong to?”
  - Semi-supervised learning - assumes a small training set and it intends to enlarge the small training set by using some unlabeled data.
  - Unsupervised learning - learns patterns from unlabelled data.
    - E.g., Clustering
    - Asks “how can I group this set of items (documents)?”
- Items can be documents, queries, emails, entities, images, etc.
- Machine learning and deep learning (e.g., RNN and LSTM) have been widely adopted for classification due to its ability to learn patterns from data rather than relying on manually crafted rules.
- **Deep learning–based text classification, which** typically uses task-oriented neural architectures (e.g., CNN, RNN, LSTM) to learn document representations from labeled data (training sets)
- **LLM-based text classification** relies on large-scale pre-trained language models (e.g., BERT) trained on massive, diverse corpora or uses prompt engineering (zero-shot/few-shot learning) for specific domains (e.g., sentiment analysis).

How to Classify or Make Decisions?

- How do humans classify items?
  - For example, suppose you had to classify the "healthiness" of a food
    - Identify set of *features* indicative of health
      - Energy, fat, protein, etc.
    - *Extract* features from foods
      - Read nutritional facts, chemical analysis, etc.
      - E.g., nutrition information panel (see right table)
    - *Combine evidence* from the features into a hypothesis
      - Add health features together to get "healthiness factor"
    - Finally, *classify* the item based on the evidence
      - If "healthiness factor" is above a certain value, then deem it healthy
- How does a machine classify items?

	Per serve
Energy	**432kJ**
Protein	2.8g
Fat	
Total	0.4g
Saturated	0.1g
Carbohydrate	
Total	18.9g
Sugars	3.5g
Fibre	6.4g
Sodium	65mg

How to Classify or Make Decisions? cont.

- Formalization of classification task
  - Let $\boldsymbol{C} = \{c_1, c_2, \ldots, c_J\}$ be a fixed set of *classes, and* $\boldsymbol{U}$ be the document space (or the set of incoming documents).
  - Let D be a training set that consists of labelled documents $<d, c>$,where $<d, c> \in \boldsymbol{U} \times \boldsymbol{C}$.
    - E.g.,

    $<d_1, c_1>$ = <"I'm so happy with my new phone", *Positive*>

    $<d_2, c_2>$ = <"I'm really disappointed with the service I received.", *Negative*>

    $<d_3, c_3>$ = <"I'm not sure if I like the new design.", *Neutral* >
  - A classifier (or classification function) maps documents to classes, the objective of the training.
    - cf: $\boldsymbol{U} \rightarrow \boldsymbol{C}$
- This type of learning is called *supervised learning* because a supervisor (the human who defines the classes and labels training documents) serves as a teacher directing the learning process.
- Please note that many other reference books may use the folowing definations:
  - Train_X, e.g., $X = [x_1, x_2, \ldots, x_m]$, x_1 = "I'm so happy with my new phone",
  - Train_y, e.g., $y = [y_1, y_2, \ldots, y_m] = [1, -1, \ldots, 0]$, where "1" means *Positive*, "-1" means *Negative* and "0" means *Neutral*.
  - where
  - $<x_j, y_j>$ means <d, c>

Evaluating Classifiers

Category c_i		Expert judgment: Yes	Expert judgment: No
Classifier judgment	Yes	TP_i	FP_i
	No	FN_i	TN_i

- Common classification metrics
  - Precision
  - Recall
  - F-measure
  - Accuracy
  - ROC curve analysis
    - The curve is created by plotting the true positive rate (TP rate) against the false positive rate (FP rate) at various threshold settings.

$$Prec = \frac{TP}{TP + FP}$$

$$Rec = \frac{TP}{TP + FN}$$

$$Acc = \frac{TP + TN}{|U|}$$

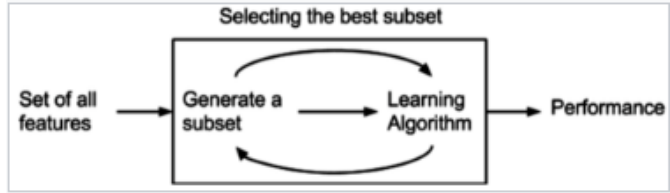
- Differences from IR metrics
  - "Relevant" replaced with "classified correctly"
  - Micro-averaging more commonly used
    - Macro-averaging: computes a simple average over classes.
    - Micro-averaging: sum per-doc decisions across classes and then computes on that sum.

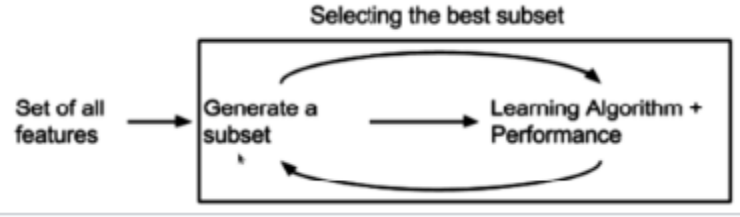
Summary of Text Classification

- **Text classification** (also known as *text categorization*) is the task of assigning text data to predefined categories using classifiers learned from labeled training samples. A classification problem can be:
  - **Binary classification**, where there are exactly two classes;
  - **Multi-class classification**, where there are more than two classes and each data object (e.g., a document) belongs to exactly one class; or
  - **Multi-label classification**, where a data object (e.g., a document) may be associated with multiple categories simultaneously.
- The text classification process using machine learning typically consists of three main stages:
  - Feature selection or feature extraction;
  - Classification model (classifier) development; and
  - Classification evaluation.
- Please note that this unit does not cover the internal architectures of deep learning models. It assumes that students already have a foundational background in machine learning or data mining. Once documents are represented in a machine-readable form (e.g., vectors or matrices), the application of machine learning models is relatively straightforward.
- Furthermore, deep learning models do not consistently outperform traditional classifiers, such as Support Vector Machines (SVM) or Naïve Bayes (NB), across all text classification tasks.
- To understand the fundamental principles of supervised decision-making, Naïve Bayes and linear SVM models are particularly valuable, as they offer substantially greater interpretability than deep learning approaches..

2. Feature Selection & Evaluation

- Text classifiers can have a very large number of features, but NOT all features are useful. Also, excessive features can increase computational cost of training and testing.
- Feature Selection Methods reduce the number of features by choosing the most useful features
  - **Filters** that select terms or patterns by ranking them with correlation coefficients.
    - They do not incorporate learning (e.g., tf*idf, BM25 weights).
  - **Wrapper methods** that assess subsets of features according to their usefulness to a given predictor.
    - They use a learning method to measure the quality of subsets of features (e.g., cross-validation) without incorporating knowledge about the specific structure of the classification or regression function.
  - **Embedded methods** implement feature selection during the model training.
    - They do not separate the learning from the feature selection part.





Other measures for assessing features

- Like Chi squared measure, Covariance (*cov*) and correlation (*cor*) also frequently used to show the difference between a variable x and the target variable y.

$$cov_{xy} = \sigma_{xy} = E[(x - \mu_x)(y - \mu_y)]$$
$$cor_{xy} = \sigma_{xy}/\sigma_x\sigma_y$$

  where, x and y are two random variables (features) with means μ_x and μ_y and standard deviation σ_x and σ_y, and E is the expected value operator.
  For example,
  x = [132,50,32,20,19,11,10,9,5]
  y = [86.7,50.7,36.0,27.9,22.8,19.3,16.7,14.7,13.2]

- They describe the degree to which two random variables or sets of random variables tend to deviate from their expected values in similar ways.

```
def cov(x,y):
    mux = sum(x)/len(x)
    muy = sum(y)/len(y)
    xd = [xi-mux for xi in x]
    yd = [yi-muy for yi in y]
    xy = [xd[i]*yd[i] for i in range(len(x))]
    return sum(xy)/(len(x)-1)


def cor(x,y):
    mux = sum(x)/len(x)
    muy = sum(y)/len(y)
    xsd = [(xi-mux)**2 for xi in x]
    ysd = [(yi-muy)**2 for yi in y]
    sx = math.sqrt(sum(xsd)/(len(x)-1))
    sy = math.sqrt(sum(ysd)/(len(y)-1))
    return cov(x,y)/(sx*sy)
```

Other measures for assessing features cont.

- **Information gain** is a commonly used feature selection measure based on information theory
  - It tells how much "information" is gained if we observe some feature.
  - It ranks features by information gain and then train model using the top *K* (*K* is typically small).
  - The general definition of information gain for a Multiple-Bernoulli Naïve Bayes classifier is computed as follows, where *H* is the entropy function:

$$\begin{aligned} IG(w) &= H(C) - H(C|w) \\ &= -\sum_{c \in \mathcal{C}} P(c) \log P(c) + \sum_{w \in \{0,1\}} P(w) \sum_{c \in \mathcal{C}} P(c|w) \log P(c|w) \end{aligned}$$

  - It can also be described as the KL divergence of the prior distribution of features *c* in *C* from the posterior distribution for features *c* given feature *w*.

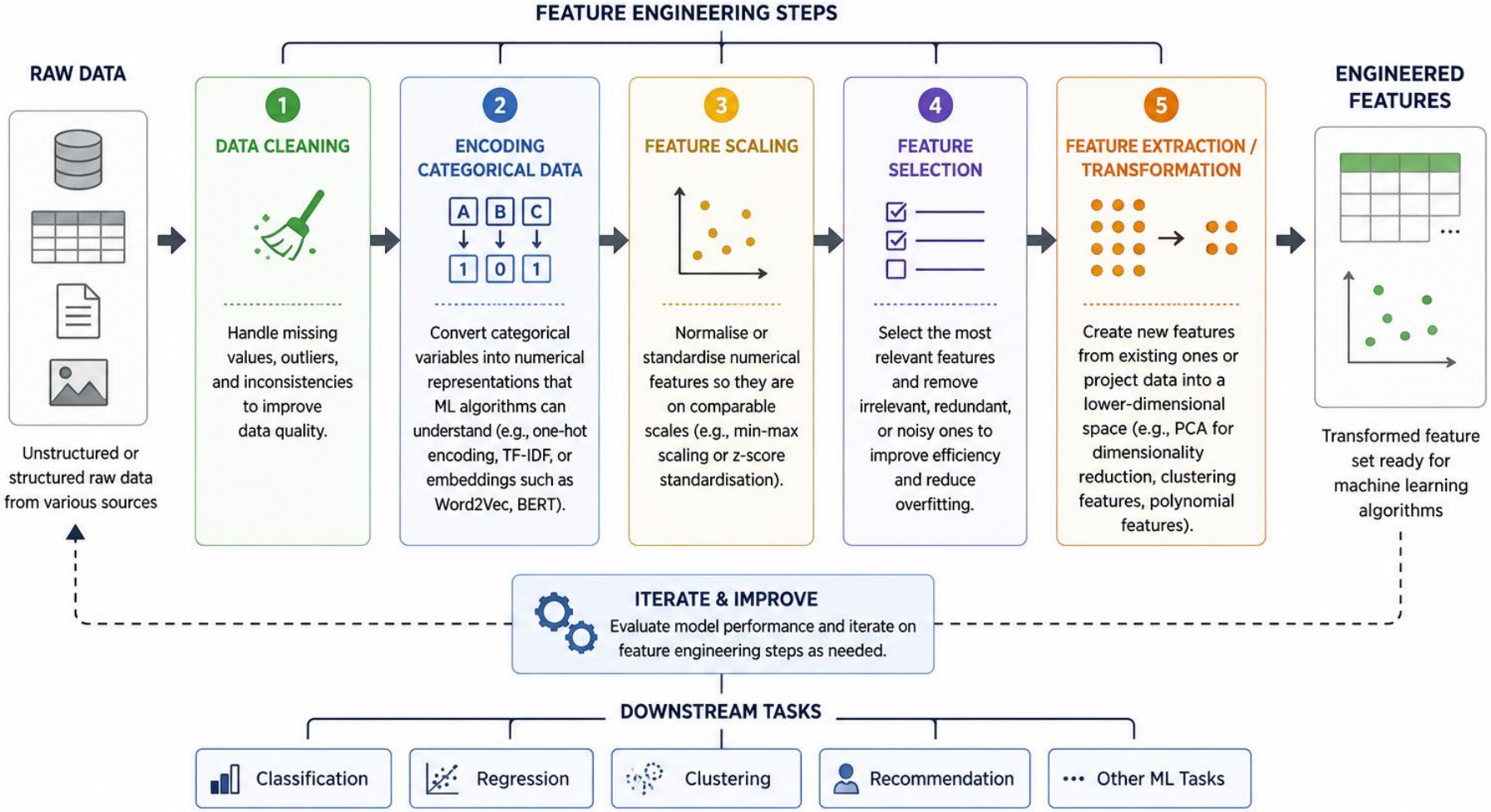
$$IG(w) = KL(P(c)||P(c|w)) = -\sum_{c \in C} P(c) log \frac{P(c)}{P(c|w)}$$

Feature Engineering

- It is the process of transforming raw data into a more effective set of features for machine learning algorithms, with the goal of improving model performance and generalization.
  - ***Data cleaning***: Handling missing values, outliers, and inconsistencies in the data to improve data quality.
  - **Encoding categorical data:** Converting categorical variables into numerical representations that machine learning algorithms can process (e.g., one-hot encoding, term weighting schemes such as tf*idf, or learned embeddings such as Word2Vec or BERT).
  - ***Feature scaling***: Normalizing or standardizing numerical features so they are on comparable scales (e.g., min-max scaling or z-score standardization), which is especially important for distance-based and gradient-based models.
  - **Feature selection:** Selecting the most relevant features while removing irrelevant, redundant, or noisy ones to improve efficiency and reduce overfitting.
  - ***Feature extraction / transformation*:** Creating new features from existing ones or projecting data into a lower-dimensional space (e.g., Principal Component Analysis (PCA) for dimensionality reduction).

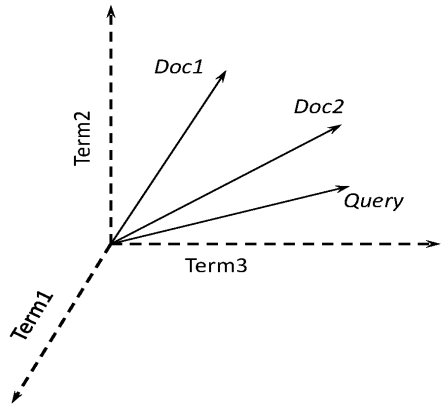
FEATURE ENGINEERING PROCESS

Feature engineering is the process of transforming raw data into a more effective set of features for machine learning algorithms, with the goal of improving model performance and generalisation.



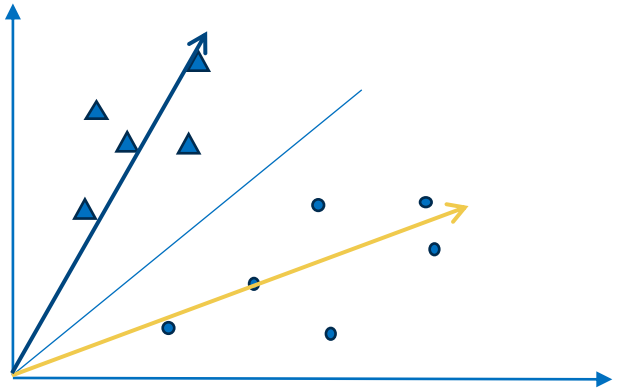
3. Vector Space Classifiers

- Rocchio classification
  - It divides the vector space into regions centered on centroids (or prototypes), one for each class, computed as the center of mass of all documents in the class.
- SVM
  - Support Vector Machines
- kNN (or k nearest neighbors)
  - For 1NN we assign each document to the class of its closest neighbor.
  - For kNN we assign each document to the majority class of its *k* closest neighbors where *k* is a parameter.
  - The rationale of kNN classification is that, based on the contiguity hypothesis, we expect a test document *d* to have the same label as the training documents located in the local region surrounding *d*.



$$\begin{array}{lcccc} & Term_1 & Term_2 & \dots & Term_t \\ Doc_1 & d_{11} & d_{12} & \dots & d_{1t} \\ Doc_2 & d_{21} & d_{22} & \dots & d_{2t} \\ \vdots & \vdots & & & \\ Doc_n & d_{n1} & d_{n2} & \dots & d_{nt} \end{array}$$

Rocchio Classifier



- For an input training set *D and* a set of class labels *C*, the training procedure of Rocchio classification uses centroids to represent classes. The centroid of a class is computed as the mean vector of class members.
- Training - given a list of vectors in each class "c_j" of *C*. We can sum all the vectors they are from class c_j to form a mean vector (or the centroid of c_j).
- Test - for any new document *d*, in the test phase, we can determine whether document *d* belongs to which class by checking which class (i.e. its centroid) is closed to *d*.

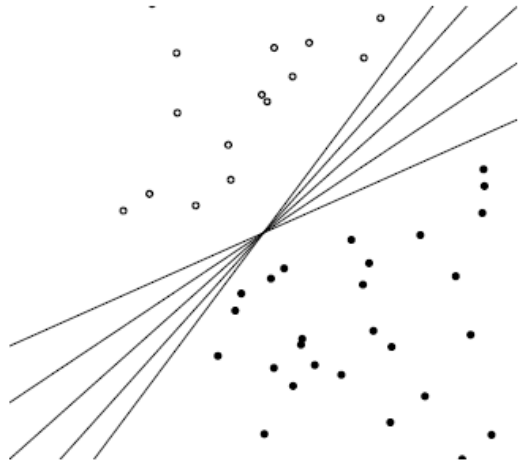
procedure TRAIN_ROCCHIO(C, D)
// C is a set of classes and D is a training set.
//Each element in D is a document-class pair (d, c).
(1) for each $c_j \in C$ do {
$D_j = \{d|(d, c_j) \in D\}$;
$\overrightarrow{\mu_j} = \frac{1}{|D_j|} \sum_{d \in D_j} \overrightarrow{d}$ }
(2) return $\{\overrightarrow{\mu_1}, \overrightarrow{\mu_2}, \cdots, \overrightarrow{\mu_{|C|}}\}$;

procedure APPLY_ROCCHIO(d, $\{\overrightarrow{\mu_1}, \overrightarrow{\mu_2}, \cdots, \overrightarrow{\mu_J}\}$)
// d is an incoming document.
(1) return $argmin_{1 \le j \le J} ||\overrightarrow{\mu_j} - \overrightarrow{d}||$;

Professor Yuefeng Li
School of Computer Science

Support Vector Machines (SVM)

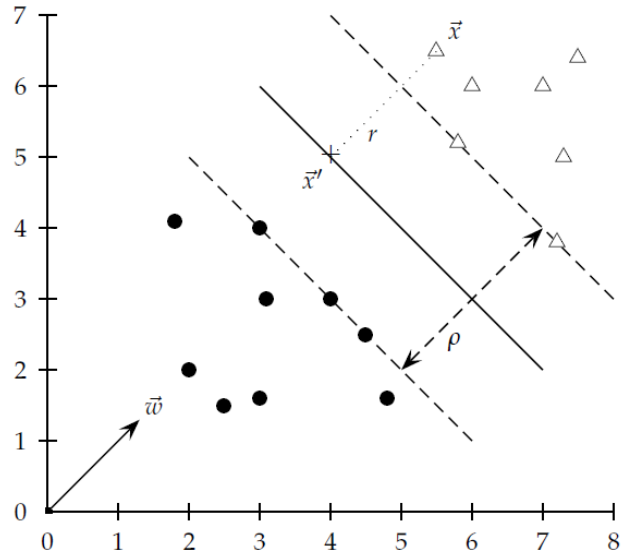
- Instead of fitting the data tightly, we prefer **a boundary** that is well-separated from all classes.
- SVM is based on geometric principles.
- Given a set of inputs labeled '+' and '-', find the "best" hyperplane that separates the '+'s and '-'s
- Questions
  - How is "best" defined? (e.g., there are lots of possible linear separators between '+'s and '-'s)
  - What if no hyperplane exists such that the '+'s and '-'s can be perfectly separated?



- SVM provides a simple idea: placing a decision boundary "in the middle" of training data often seems reasonable
- However, the best boundary for training data may not generalize well to test data
- We need a boundary that is not just accurate, but also robust to unseen data.

SVM cont.

- SVM defines a decision surface that is maximally far from any data point
- The closest points to the decision boundary determine the margin
- The margin is the distance between the decision surface and the nearest data points
- Only a small subset of training data determines the decision boundary
- These points are called **support vectors** that are closest to the decision boundary.
- The SVM objective is to:
  - **Maximize the margin to improve generalization performance**

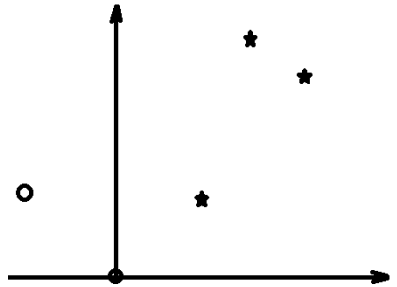


- SVM first defines an optimization problem to search for the hyperplane, and then assign a weight α_i to each training data point i.
- It also identifies support vectors, data points with $\alpha_i > 0$.
- It also constructs a decision function using a kernel function and these support vectors.

SVM Tools

- Solving SVM optimization problem is not straightforward
- Many good software packages exist
  - SVM-Light
  - LIBSVM
  - R library
  - Matlab SVM Toolbox
  - scikit-learn (Machine Learning in Python)

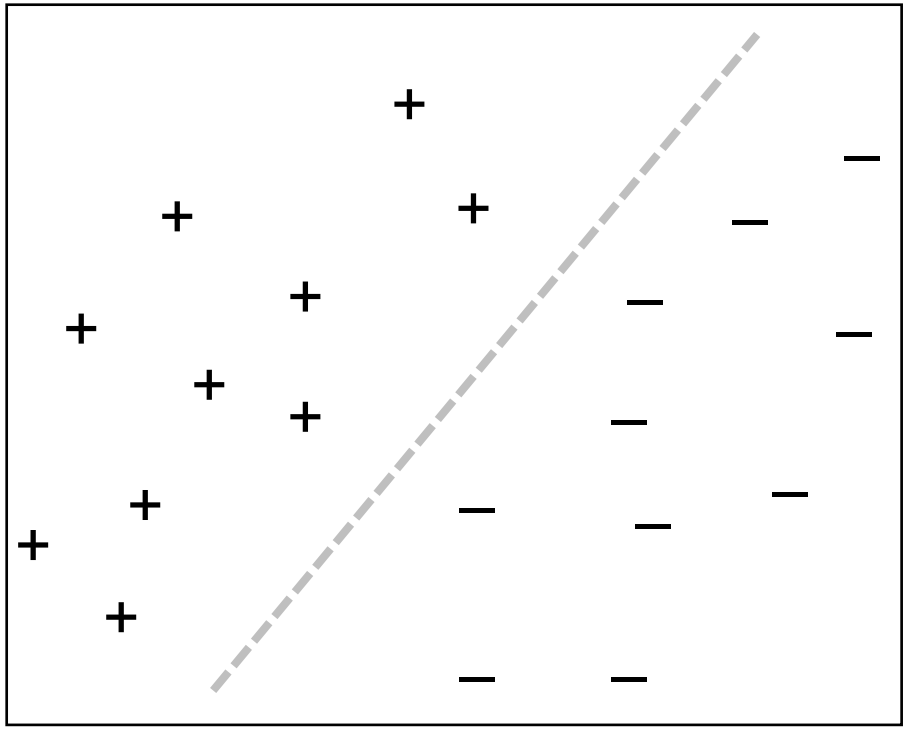
Example of scikit-learn for SVM



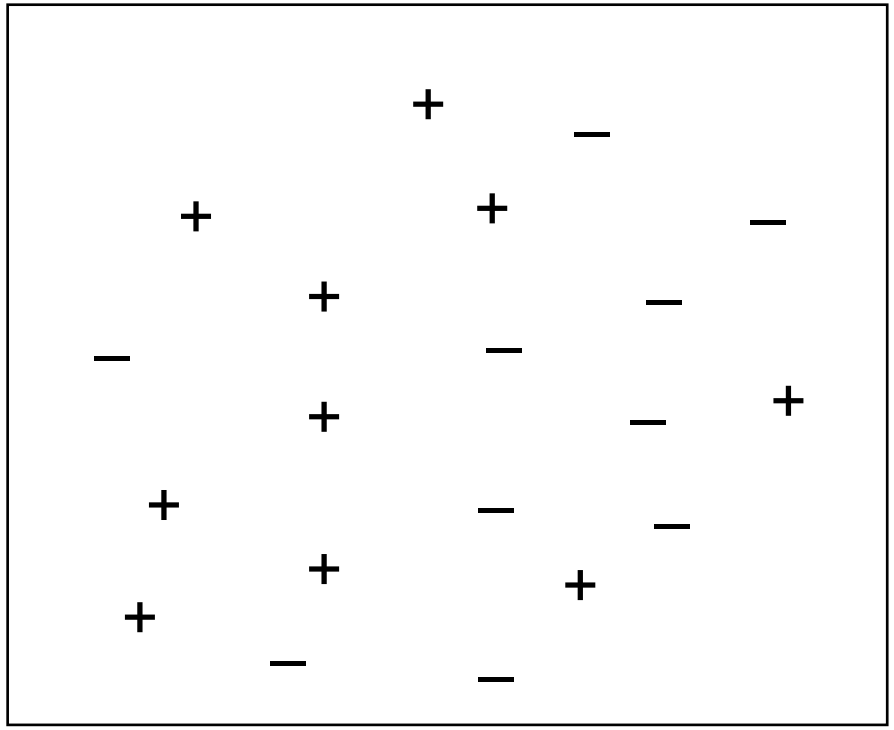
```
from sklearn import svm
X = [[0, 0], [1, 1], [2, 2], [-1, 1], [1.5, 2.5]]
y = [0, 1, 1, 0, 1]
clf = svm.SVC(gamma='scale')
clf.fit(X, y)
result1 = clf.predict([[2., 2.], [3., 3.], [-2., 0]])
print("SVC Predication Results: %s" % result1)
supp_v = clf.support_vectors_
print("Support Vectors: %s" % supp_v)
lin_clf = svm.LinearSVC()
lin_clf.fit(X, y)
result2 = clf.predict([[2., 2.], [3., 3.], [-2., 0]])
print("Linear Predication Results: %s" % result2)
dec = lin_clf.decision_function([[2., 2.], [3., 3.], [-2., 0]])
print("Linear decision function values: %s" % dec)
```

```
========== RESTART: C:\Python27\svm1test.py ==========
SVC Predication Results: [1 1 0]
Support Vectors: [[ 0.   0. ]
 [-1.   1. ]
 [ 1.   1. ]
 [ 2.   2. ]
 [ 1.5  2.5]]
Linear Predication Results: [1 1 0]
Linear decision function values: [ 1.91577867  3.13682358 -2.12630714]
>>>
```

Separable vs. Non-Separable Data



Separable



Non-Separable

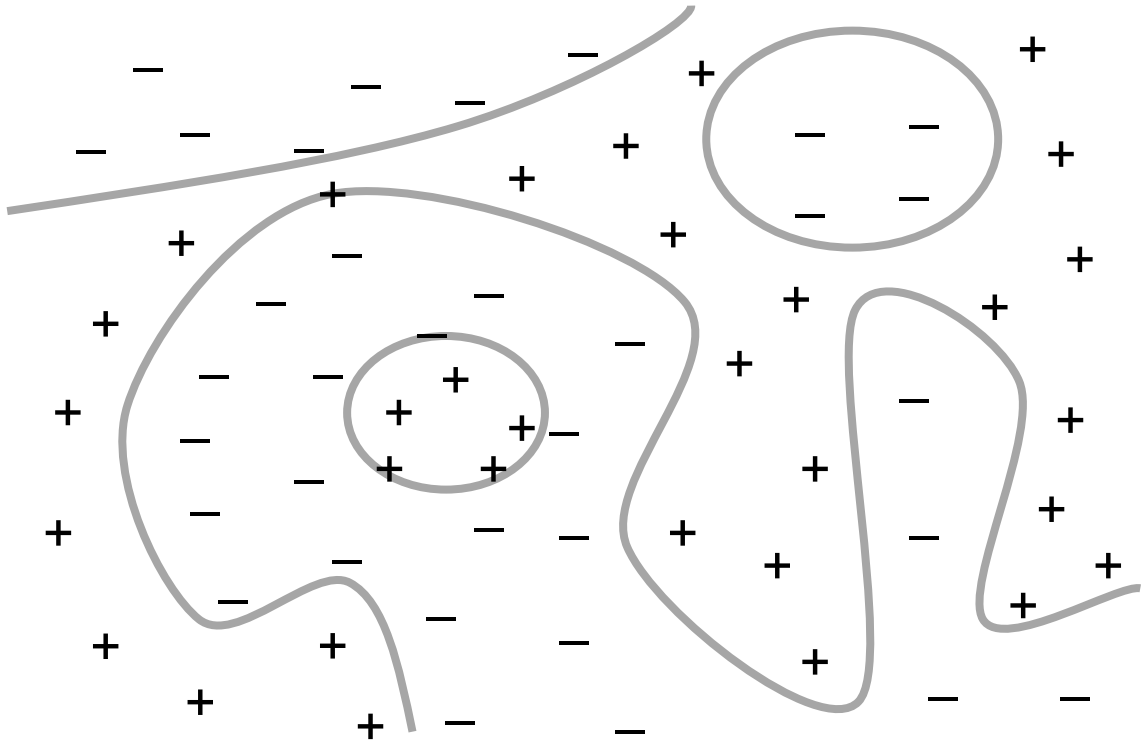
kNN training and testing

- The k-nearest neighbors algorithm (k-NN) is a non-parametric supervised learning method.
- Given an incoming document, the nearest neighbour classifier finds the training example that is nearest (according to some distance metric) to it.

procedure TRAIN-KNN(C, D)
1. $D' \leftarrow$ PRE_PROCESS(D)
2. $k \leftarrow$ *SELECT-K*(C, D')
3. **return** D', k

procedure APPLY-KNN(C, D', k, d)
1. $S_k \leftarrow$ *COMPUTE_NEAREST_NEIGHBORS*(D', k, d)
2. **for** each $c_j \in C$ **do**
3. $p_j \leftarrow |S_k \cap c_j|/k$
4. **return** $\text{argmax}_j\, p_j$ # where j is a class label. The return value is the class label that maximizes p_j.

Example of Nearest Neighbor Classification



- The pluses + and minuses - indicate positive and negative training examples, respectively.
- The solid grey line shows the actual decision boundary, which is highly non-linear.
- SVMs with a non-linear kernel, would have a difficult time fitting a model to this data.
- For this reason, the nearest neighbour classifier is optimal.

4. Generative Model - Naïve Bayes Classifier

- Naïve Bayes (NB) is one of the most straightforward yet effective classification techniques.
- Bayes decision - it uses Bayes' rule to classify documents into two classes, e.g., relevant class and non-relevant class.
- Probabilistic classifer based on Bayers' rule:

$$P(c|d) = \frac{P(d|c)P(c)}{\sum_{c_i \in C} P(d|c = c_i)P(c = c_i)}$$

where, c is a random variable corresponding to the class
d is a random variable corresponding to the input document.

Professor Yuefeng Li
School of Computer Science

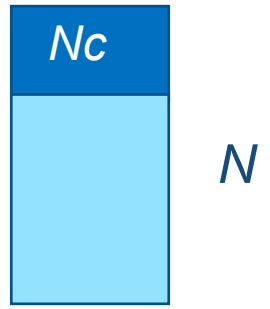
Naïve Bayes Classifier cont.

- Documents are classified according to an *argmax* function - means return the class c, out of classes C, that maximizes $P(c \mid d)$.
- Two steps to interpret Class(d)
  1. max_P = max([$P(c|d)$ for c in C])
  2. Find c in C such that $P(c|d) = max_P$
- Must estimate $P(d \mid c)$ and $P(c)$
  - $P(c)$ is the probability of observing class c
  - $P(d \mid c)$ is the probability that document d is observed given the class is known to be c.
- $P(c)$ Estimated as the proportion of training documents in class c:
  - N_c is the number of training documents with class label c
  - N is the total number of training documents.

$$\text{Class}(d) = \arg\max_{c \in C} P(c|d)$$

$$= \arg\max_{c \in C} \frac{P(d|c)P(c)}{\sum_{c \in C} P(d|c)P(c)}$$

$$P(c) = \frac{N_c}{N}$$



Estimating $P(d \mid c)$

- Estimate depends on the *event space* used to represent the documents.
- **Multiple Bernoulli Event Space**
  - Documents are represented as binary vectors
    - One entry for every word in the vocabulary
    - Entry $i = 1$ if word i occurs in the document and is 0 otherwise
  - Multiple Bernoulli distribution is a natural way to model distributions over binary vectors.
  - Same event space as used in the classical probabilistic information retrieval model.

Example 1. Multiple Bernoulli Document Representation

document *id*	cheap	buy	banking	dinner	the	*class*
1	0	0	0	0	1	not spam
2	1	0	1	0	1	spam
3	0	0	0	0	1	not spam
4	1	0	1	0	1	spam
5	1	1	0	0	1	spam
6	0	0	1	0	1	not spam
7	0	1	1	0	1	not spam
8	0	0	0	0	1	not spam
9	0	0	0	0	1	not spam
10	1	1	0	1	1	not spam

$D = \{d_1, d_2, \ldots, d_{10}\}$, $C = \{c_1, c_2\} = \{$'spam', 'not spam'$\}$,

$V = \{w_1, w_2, \ldots, w_5\} = \{$'cheep', 'buy', 'banking', 'dinner', 'the'$\}$

$Nc_1 = 3$, $Nc_2 = 7$. $P(c_1) = 3/10 = 0.30$, $P(c_2) = 7/10 = 0.70$

Multiple-Bernoulli: Estimating *P*(*d* | *c*)

- $P(d \mid c)$ is computed as:

$$P(d|c) = \prod_{w \in \mathcal{V}} P(w|c)^{\delta(w,d)} \left(1 - P(w|c)\right)^{1-\delta(w,d)}$$

  - where $\delta(w, d)$ is 1 if and only if term w occurs in document d, otherwise it is 0.
- Laplacian smoothed estimate:

$$P(w|c) = \frac{df_{w,c} + 1}{N_c + 1}$$

- Collection smoothed estimate:

$$P(w|c) = \frac{df_{w,c} + \mu \frac{N_w}{N}}{N_c + \mu}$$

  - Where $df_{w,c}$ is the number of training documents with class label c in which term w occurs, N_w is the total number of training documents in which term w occurs, and μ is a single tuneable parameter.

Example 1 cont.

- For $d4 = [1, 0, 1, 0, 1]$, $c1$ = 'Spam'; based on the 1st Equation, we have
  $P(d4|c1) = P(w1|c1)*(1-P(w2|c1))*P(w3|c1)*(1-P(w4|c1))*P(w5|c1)$
- Also, $df_{w1,c1} = 3$, $N_{w1} = 4$, $N_{c1} = 3$, $N = 10$, and let $\mu = 0.5$
- Then by using the collection smoothed estimate, $P(w1|c1) = [3 + 0.5* (4/10)] / [3+0.5] = 0.914$

Multinomial Event Space

- Documents are represented as vectors of term frequencies
  - One entry for every word in the vocabulary
  - Entry i = number of times that term i occurs in the document
- Multinomial distribution is a natural way to model distributions over frequency vectors
- Same event space as used in the language modeling retrieval model

Example 2. Multinomial Document Representation

document *id*	cheap	buy	banking	dinner	the	*class*
1	0	0	0	0	2	not spam
2	3	0	1	0	1	spam
3	0	0	0	0	1	not spam
4	2	0	3	0	2	spam
5	5	2	0	0	1	spam
6	0	0	1	0	1	not spam
7	0	1	1	0	1	not spam
8	0	0	0	0	1	not spam
9	0	0	0	0	1	not spam
10	1	1	0	1	2	not spam

$D = \{d_1, d_2, \ldots, d_{10}\}$, $C = \{c_1, c_2\} = \{$'spam', 'not spam'$\}$,

$C_1 = \{d_2, d_4, d_5\}$, $C_2 = \{d_1, d_3, d_6, d_7, \ldots, d_{10}\}$.

$V = \{w_1, w_2, \ldots, w_5\} = \{$'cheep', 'buy', 'banking', 'dinner', 'the'$\}$

$|c_1| = 20$, $|c_2| = 15$, $|C| = |c_1| + |c_2| = 35$, $|V| = 5$.

Where $|c|$ is the total number of times of terms that occur in training documents with class label c.

Multinomial: Estimating P(*d* | *c*)

- $P(d \mid c)$ is computed as:

$$P(d|c) = P(|d|)\,(tf_{w_1,d}, tf_{w_2,d}, \ldots, tf_{w_{\mathcal{V}},d})!\prod_{w\in\mathcal{V}} P(w|c)^{tf_{w,d}}$$

$$\propto \prod_{w\in\mathcal{V}} P(w|c)^{tf_{w,d}}$$

- Laplacian smoothed estimate:

$$P(w|c) = \frac{tf_{w,c}+1}{|c|+|\mathcal{V}|}$$

- Collection smoothed estimate:

$$P(w|c) = \frac{tf_{w,c}+\mu\frac{cf_w}{|C|}}{|c|+\mu}$$

  - where $tf_{w,c}$ is the number of times that term w occurs in class c in the training set, and $|c|$ is the total number of times of terms that occur in training documents with class label c.

Example 2 cont.

- For $d4$ = [2, 0, 3, 0, 2] , $c1$ = 'Spam'; we have

  $tf_{w1,d4}$= 2, $tf_{w2,d4}$= 0, $tf_{w3,d4}$= 3, $tf_{w4,d4}$= 0, $tf_{w5,d4}$= 2

- Based on the 1st Equation here, we have

  $P(d4|c1) = (P(w1|c1))^2 * (P(w3|c1))^3 * P(w5|c1)^2$

- Also, $tf_{w1,c1}$ = 10, $|c1|$= 20, $|V|$ =5
- Then by using the Laplacian smoothed estimate, $P(w1|c1) = (10+1) / (20+5) = 0.44$

NB implementation

- Let document d be represented as a vector $d=(w_1, w_2, \ldots, w_n)$. We have

$$P(c|d)P \propto P(c)\prod_{w_i \in d} P(w_i|c)$$

  - where each conditional probability $P(w_i|c)$ indicates how likely term w_i is relevant to class c, and ∝ (\propto symbol) denotes proportionality.
- The formula above can be converted to a summation by applying the logarithm to both sides

$$P(c|d) \propto \log\big(P(c)\big) + \sum_{w_i \in d} \log(P(w_i|c))$$

Naive Bayes Algorithm (multinomial model): Training

procedure TRAIN_MULTINOMIAL_NB(C, D)

1. $V \leftarrow EXTRACT\_VOCABULARY(D)$
2. $N \leftarrow COUNT\_DOCS(D)$
3. **for each** $c \in C$ **do** {
4. $Nc \leftarrow COUNT\_DOCS\_IN\_CLASS(D, c)$
5. $prior[c] \leftarrow Nc/N$
6. $text_c \leftarrow CONCATENATE\_TEXT\_OF\_ALL\_DOCS\_IN\_CLASS(D, c)$
7. **for each** $w \in V$ **do**
8. $tf_{w,c} \leftarrow COUNT\_TOKENS\_OF\_TERM(text_c, t)$
9. **for each** $w \in V$ **do**
10. $condprob[w][c] \leftarrow (tf_{w,c}+1) / (|c| + |V|)$ }
11. **return** V, $prior$, $condprob$

Naive Bayes Algorithm (multinomial model): Testing

procedure APPLY_MULTINOMIAL_NB(C,V, *prior*, *condprob*, d)

1. $W \leftarrow$ *EXTRACT_TOKENS_FROM_DOC*(V, d)
2. **for** each $c \in C$ **do**
3. $\quad$ *score*[c] $\leftarrow$ *log prior*[c]
4. $\quad$ **for** each $w \in W$ **do**
5. $\quad\quad$ *score*[c] += *log condprob*[w][c]
6. **return** $\text{argmax}_{c \in C}$ *score*[c]

5. Sentiment Analysis

- **Sentiment analysis** (**opinion mining**) discovers users' opinions about products or services in on-line reviews or feedback, or observes trends in public mood (e.g., analysis of clinical records).
- It is widely used to voice of customers (users) materials such as reviews and survey responses, online and social media, and healthcare materials for applications that range from marketing to customer service to clinical medicine.
- Opinion can be represented as a tuple of Entity, Aspect, Orientation, Opinion Holder and Time.
  - An **entity** is the name of an entity, which could refer to a product for example (e.g., "iphone 16 pro max").
  - An **aspect** can be a feature, component or function of the entity (e.g., battery capacity, clock speed, body material, broadband generation, cellular network, general, etc.).
  - The **orientation** is the opinion provided about the entity and/or the aspect that was provided by the opinion holder at a specific time.

Tasks in Sentiment Analysis

Polarity classification

- Group the expressed opinion in a document, a sentence or an entity feature/aspect in positive, negative or neutral regions.

Subjectivity classification

- Its goal is to separate subjective from objective information, a binary classification task.
- It is regarded as a prerequisite to sentiment polarity classification. It may be tackled at different levels of granularity. For instance,
  - At the document level the aim is to distinguish review-like documents from non-review documents or factual newspaper articles from editorial comments.
  - On a more fine-grained level, the task is to identify individual text passages (e.g., sentences) as being subjective or objective.

Emotion classification

- The goal is to classify a piece of text according to a predefined set of basic emotions.
- It tries to identify more fine-grained differences in the expression of sentiment, e.g., six “basic” emotions - anger, disgust, fear, happiness, sadness, and surprise.

Tasks in Sentiment Analysis cont.

Source detection

- It aims to identify the person, organisation, or more generally, the entity that is the source of subjective information, including named entity recognition and relationship extraction.
- It is an information extraction task.
- A typical application for sentiment source detection is a multi-perspective question answering system that tries to answer questions of the form: “What is X’s viewpoint/opinion on topic Y?”
- It is an information extraction task.

Target detection

- The goal of sentiment target detection is to determine the subject of a sentiment expression. For example,
  - Which blogs report positively and which negatively on the topic of settlement policy?
- It is a problem of information retrieval, where sentences or documents are classified or ranked according to their relevance towards a given topic or a question.
  - E.g., the most recent google-released Bidirectional Encoder Representations from Transformers (**BERT**) https://arxiv.org/abs/1810.04805

Aspect-based Sentiment Analysis

- Sentiment analysis also includes more complex tasks such as the Aspect-Based Sentiment Analysis and Multifaceted Analysis of Subjective Texts that focuses on specific sentiment or opinion phenomena.
- Aspects are nouns and/or noun phrases, for example, “face recognition”, “zoom”, and “touch screen” are aspects of the product “camera”. They can be used to analyze sentiment and opinion information at the more fine-grained aspect level.
- Opinion words are mostly adjectives. They are the closest adjective to the aspects in the sentence. An opinion lexicon can be used to identify and extract opinion words along with their orientation.
- **Extraction of opinions**:
    1) Build a list of aspects from two sources: **product specifications** and **word synonyms**. Product specifications is a list provided by the manufacturer for each product, while synonyms are the matching words taking from the WordNet dictionary (or stem classes).
    2) Identify the aspect and opinion tags in sentences. You may need to group aspects based on frequency and synonyms, or use
        - pattern mining to find frequent sets of POS tags that occur together. A set of POS tags is defined as frequent pattern if it appears in more than 1% (minimum support) of the review sentences.

            For example, the tag [NN] of aspect appears first, then follow by a sequence of tags [VBZ][RB][JJ], e.g., the sentence “software is absolutely terrible”.
    3) Weights are assigned to tags.
    4) Weighting sentences by adding tags’ weights and then select sentences with high scores.

Adjective, adverb and verb weights

- Weights can be assigned to adjective, adverb, and verb tags to indicate the strength and direction of sentiment.
- These weights, often based on dictionaries or machine learning models, help determine the overall sentiment of a sentence or text.

Tags	Description	Weight
JJ	Adjective	1
JJR	Comparative Adjective	2
JJS	Superlative Adjective	3
RB	Adverb	1
RBR	Comparative Adverb	2
RBS	Superlative Adverb	3

- Note that categories are used for verbs.
- If the sentence contains a verb from positive categories, then “+1” will be added to the weight.
- If the verb is from negative categories, then “-1” will be subtracted from the total weight.

Verb category	Orientation	Verbs	Comments
Tell verbs	Positive	tell	Positively reinforce an opinion
Chitchat verbs	Positive	argue, chatter, gab	Positively reinforce opinion is being expressed
Advise verbs	Positive	advise, instruct	Positively reinforce an opinion
	Negative	admonish, caution, warn	Negatively reinforce the degree of certainty about a given opinion

6. Methodologies for Sentiment Analysis

- Lexicon-based analysis
  - It uses a set of predefined rules and heuristics to determine the sentiment of a piece of text.
- Pre-trained transformer-based deep learning
  - A deep learning-based approach, as seen with BERT and GPT-4,
  - However, these models require significant computational resources and may not be practical for all use cases.
- Machine learning
  - Training a model to identify the sentiment of a piece of text based on a set of labeled training data.
- LLM-based Sentiment Analysis

	reviewText	Positive
0	This is a one of the best apps acording to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1
...	...	...
19995	this app is fricken stupid.it froze on the kin...	0
19996	Please add me!!!!! I need neighbors! Ginger101...	1
19997	love it! this game. is awesome. wish it had m...	1
19998	I love love love this app on my side of fashio...	1
19999	This game is a rip off. Here is a list of thin...	0

20000 rows × 2 columns

Example of Amazon reviews

Example of Amazon Reviews & text pre-processing

```
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

def parse_text(text):
    # Tokenize the text
    tokens = word_tokenize(text.lower())
    # Remove stop words
    filtered_tokens = [token for token in tokens if token not in stopwords.words('english')]
    # Lemmatize the tokens
    lemmatizer = WordNetLemmatizer()
    lemmatized_tokens = [lemmatizer.lemmatize(token) for token in filtered_tokens]
    # Join the tokens back into a string
    processed_text = ' '.join(lemmatized_tokens)
    return processed_text
```

	reviewText	Positive
0	This is a one of the best apps acording to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1
...	...	...
19995	this app is fricken stupid.it froze on the kin...	0
19996	Please add me!!!!! I need neighbors! Ginger101...	1
19997	love it! this game. is awesome. wish it had m...	1
19998	I love love love this app on my side of fashio...	1
19999	This game is a rip off. Here is a list of thin...	0

20000 rows × 2 columns

The Natural Language Toolkit (NLTK) Library

- NLTK is a popular open-source library for natural language processing (NLP) in Python.
- Load a dataset of Amazon reviews using pd.read_csv() (labelled dataset). This will create a DataFrame object *df*.
- After using the function *parse_text*(), initialize NLTK sentiment analyzer, and define function cal_sentiment_score().
- At last, evaluate the proposed get_sentiment() function by comparing two columns 'Positive' and 'sentiment'.

```
import nltk
df = pd.read_csv('https://raw.githubusercontent.com/pycaret/pycaret/master/datasets/amazon.csv')
# apply the parse_text function via df
df['reviewText'] = df['reviewText'].apply(parse_text)
analyzer = SentimentIntensityAnalyzer()
def cal_sentiment_score(text):
    theta = 0.15 # a threshold
    scores = analyzer.polarity_scores(text)
    sentiment = 1 if scores['pos'] > theta else 0
    return sentiment
# apply cal sentiment score function
df['sentiment'] = df['reviewText'].apply(cal_sentiment_score)
from sklearn.metrics import confusion_matrix
print(confusion_matrix(df['Positive'], df['sentiment']))
from sklearn.metrics import classification_report
print(classification_report(df['Positive'], df['sentiment']))
```

```
[[ 2059  2708]
 [ 1545 13688]]
              precision    recall  f1-score   support

           0       0.57      0.43      0.49      4767
           1       0.83      0.90      0.87     15233

    accuracy                           0.79     20000
   macro avg       0.70      0.67      0.68     20000
weighted avg       0.77      0.79      0.78     20000
```

The Natural Language Toolkit (NLTK) Library cont.


```
# apply the parse_text function via df
df['reviewText'] = df['reviewText'].apply(parse_text)
df
```

	reviewText	Positive
0	one best apps acording bunch people agree bomb...	1
1	pretty good version game free . lot different ...	1
2	really cool game . bunch level find golden egg...	1
3	silly game frustrating , lot fun definitely re...	1
4	terrific game pad . hr fun . grandkids love	1
...	...	...
19995	app fricken stupid.it froze kindle wont allow ...	0
19996	please add ! ! ! ! ! need neighbor ! ginger101...	1
19997	love ! game . awesome . wish free stuff house ...	1
19998	love love love app side fashion story fight wo...	1
19999	game rip . list thing make better & bull ; fir...	0

20000 rows × 2 columns

```
# apply get_sentiment function
df['sentiment'] = df['reviewText'].apply(cal_sentiment_score)

df
```

	reviewText	Positive	sentiment
0	one best apps acording bunch people agree bomb...	1	1
1	pretty good version game free . lot different ...	1	1
2	really cool game . bunch level find golden egg...	1	1
3	silly game frustrating , lot fun definitely re...	1	1
4	terrific game pad . hr fun . grandkids love	1	1
...	...	...	...
19995	app fricken stupid.it froze kindle wont allow ...	0	0
19996	please add ! ! ! ! ! need neighbor ! ginger101...	1	1
19997	love ! game . awesome . wish free stuff house ...	1	1
19998	love love love app side fashion story fight wo...	1	1
19999	game rip . list thing make better & bull ; fir...	0	1

20000 rows × 3 columns

7. Sentiment Analysis in the Era of LLMs

- large language models (LLMs) have demonstrated impressive performance on various NLP tasks, including sentiment analysis.
- They can directly perform tasks in zero-shot or few-shot in context learning manner and achieve strong performance without the need for any in domain supervised training.
- It looks that LLMs already show strong sentiment analysis ability in zero-shot settings [6].
- But when it comes to more complex tasks such as ABSA (Aspect-Based Sentiment Analysis) tasks that require structured sentiment information, or MAST (Multifaceted Analysis of Subjective Texts) tasks requiring a deep understanding of specific sentiment phenomena, LLMs still lag behind SLMs trained with in-domain data [6].
- With a limited quantity of annotated data under the few-shot setting, LLMs with in-context learning consistently outperform SLMs trained with the same amount of data for all types of tasks.
  - This suggests that the application of LLMs is advantageous when annotation resources are scarce.

Example prompts of LLM-based text classification for Sentiment Analysis

1. Zero-shot Text Classification Prompt

Use case: No labeled training data available.

Prompt:

```vbnet
You are a text classification system.

Task: Classify the sentiment of the following text into one of the categories:
[Positive, Neutral, Negative].

Text:
"The methodology looks promising, but the experimental results are disappointing."

Output only the label.
```

Expected output:

```mathematica
Negative
```

2. Few-shot Text Classification Prompt

Use case: Improve consistency with a small number of examples.

Prompt:

```vbnet
You are a text classification system.

Task: Classify each text into one of the categories:
[Positive, Neutral, Negative].

Examples:
Text: "The paper is well written and the results are impressive."
Label: Positive

Text: "The approach is standard and follows previous work."
Label: Neutral

Text: "The experiments are flawed and poorly designed."
Label: Negative

Now classify the following text:
"The methodology is elegant, but the results are not convincing."

Output only the label.
```

Expected output:

```mathematica
Negative
```

Example Prompts for More Complex Sentiment Analysis

Aspect-Based Sentiment Analysis

Please perform Unified Aspect-Based Sentiment Analysis task. Given the sentence, tag all (aspect, sentiment) pairs. Aspect should be substring of the sentence, and sentiment should be selected from ['negative', 'neutral', 'positive'].

If there are no aspect-sentiment pairs, return an empty list; otherwise return a python list of tuples containing two strings in single quotes.

Please return python list only, without any other comments or texts.

Sentence: We discuss their opinions about the new stadium. Label: []

Sentence: The new stadium is beautiful, and the design is inspired by the Queenslander architecture style. Label: [('stadium', 'positive'), ('design', 'positive')]

Sentence: What I care about is the return on investment and whether this money can be better used for other public services.

Label:

[('return on investment', 'negative'), ('this money', 'negative')]

Multifaceted Analysis of Subjective Texts

Please perform Hate Detection task. Given the sentence, assign a sentiment label from ['hate','non-hate'].

Return label only without any other text.

Sentence: I must take the bus to work due to limited parking availability on campus. Label: non-hate

Sentence: Thanks to our great prime minister, haha, our homeless still sleep on the street. Label: hate

Sentence: I go outside, feel miserable and burnt.

Label:

non-hate

References

[1] chapter 9 (Section 9.1) in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.

[2] Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, *An Introduction to Information Retrieval*, Cambridge University Press, 2009.

[3] S-T Li and F-C Tsai, A fuzzy conceptualization model for text mining with application in opinion polarity classification, *Knowledge-based systems*, 39(2013), 23-33.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature 521, pp. 436–44, 2015.

[5] M. Hady, F. Schwenker, "Semi-supervised learning," in Handbook on Neural Info. Proce., Springer, 2013, 215–239.

[6] Wenxuan Zhang, Yue Deng, Bing Liu, Sinno Pan, and Lidong Bing. 2024. Sentiment Analysis in the Era of Large Language Models: A Reality Check. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3881–3906.

[7] scikit-learn is an open source python library and tools for data mining.

- home page:
  - https://scikit-learn.org/stable/index.html
- installation:
  - https://scikit-learn.org/stable/install.html
- svm:
  - https://scikit-learn.org/stable/modules/svm.html